1. Fie un graf G neponderat pe care se aplica un BFS pornind din nodul S. Notam cu d(S,u) si d(S,v) lungimile celor mai scurte cai gasite de BFS. Daca u este descoperit de BFS inainte de v, care din urmatoarele afirmatii este adevarata?

Alegeți o opțiune:

- a. d(S,u) >= d(S,v)
- b. d(S,u) < d(S,v)
- c. d(S,u) <= d(S,v) ●
- d. d(S,u) > d(S,v)

Feedback

Răspunsul corect este: d(S,u) <= d(S,v)

2. Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care dintre următoarele secvențe de pași sunt complete și plauzibil a fi aplicate de algoritmul AC1 ?

Alegeți o opțiune:

- a. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)
- b. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B) ●
- c. REVISE(A, B), REVISE(B, C)
- d. REVISE(A, B), REVISE(B, C), REVISE(A, B), REVISE(B, C)

Feedback

Răspunsul corect este: REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)

3. Care din următoarele funcții de tip euristică NU este admisibilă pentru algoritmul A* care calculează ruta de distanță minimă pe calea ferată între două orașe, o sursă și o destinație? Pentru toate funcțiile euristică de mai jos, valoarea euristicii pentru destinație este egală cu 0, valorea fiind calculată doar pentru restul nodurilor.

Alegeți una sau mai multe opțiuni:

- [ ] a. o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație
- [ ] b. o valoare constantă pentru fiecare nod, egală cu valoarea muchiei de cost minim din graf (distanța minimă pe cale ferată între cele mai apropiate două orașe)
- [x] c. lungimea în linie dreaptă de la fiecare nod la destinație
- [x] d. lungimea rutei deja parcurse până la fiecare nod

Feedback

The correct answers are: lungimea rutei deja parcurse până la fiecare nod, o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație

4. Pentru care dintre următoarele distribuții de probabilitate de apariție a cheilor vom avea cel puțin doi arbori Huffman distincți?

Alegeți una sau mai multe opțiuni:

- [x] a. 1/16, 1/16, 1/8, 1/4, 1/2
- [x] b. 1/16, 1/16, 1/8, 1/8, 1/4, 3/8
- [ ] c. 1/20, 1/10, 1/10, 3/20, 7/20, 1/4
- [ ] d. 1/4, 1/4, 1/4, 1/4

Feedback

The correct answers are: 1/16, 1/16, 1/8, 1/8, 1/4, 3/8, 1/4, 1/4, 1/4, 1/4, 1/20, 1/10, 1/10, 3/20, 7/20, 1/4

5. Dându-se un vector de n numere sortat crescător, fie t(n) complexitatea celui mai eficient algoritm care verifică dacă există 2 elemente în vector a căror sumă să fie mai mică de 100. Care din afirmațiile de mai jos este adevărată?

Alegeți o opțiune:

a. n * log(n) <= t(n) < n/2

b. t(n) apartine O(1)

c. t(n) = n/2

d. n <= t(n) <= n * log(n)

Feedback

Răspunsul corect este: t(n) apartine O(1)

6. Într-o sesiune de examene ce durează n zile trebuie date m examene (m<n). Profesorii materiilor respective impun condiționări legate de momentul maxim de timp până la care pot fi date examenele. Cum poate fi rezolvată cel mai eficient problema planificării unui număr maxim posibil de examene?

Alegeți o opțiune:

a. folosind algoritmi aleatori, se generează diverse configurații până când se nimerește una validă

b. folosind greedy, se sortează crescător condițiile impuse de profesori și se stabilesc zilele de examen astfel încât să se respecte condițiile

c. folosind backtracking, se generează toate variantele și se alege cea care respectă cât mai multe condiționări

d. folosind programare dinamică, se generează un tablou care să rețină toate examenele ce pot fi date până în ziua i și apoi se verifică câte examene se pot da până în ziua n

Feedback

Răspunsul corect este: folosind greedy, se sortează crescător condițiile impuse de profesori și se stabilesc zilele de examen astfel încât să se respecte condițiile

7. Un algoritm A* (cu o euristică admisibilă) este întotdeauna:

Alegeți o opțiune:

- a. nici complet nici optimal
- b. optimal
- c. complet
- d. complet și optimal (selected)

Feedback

Răspunsul corect este: complet și optimal

8. Intr-un graf G neorientat se aplica algoritmul DFS si se obtine o padure de arbori DFS T. Fie u un nod din G si v primul nod alb vizitat de DFS dupa u. Care dintre afirmatiile de mai jos este intotdeauna corecta?

Alegeți o opțiune:

- a. Daca (u, v) nu este muchie in G, atunci u si v au acelasi parinte in T
- b. (u, v) este muchie in G si u este descendent al lui v in T
- c. Daca (u, v) nu este muchie in G, atunci u este frunza in T (selected)
- d. (u, v) este muchie in G si v este descendent al lui u in T

Feedback

Răspunsul corect este: Daca (u, v) nu este muchie in G, atunci u este frunza in T

9. Care este numărul maxim posibil de arbori de parcurgere în adâncime a unui graf orientat conex (dacă considerăm graful neorientat, dar nu obligatoriu tare conex) cu n noduri?

Alegeți o opțiune:

- a. n/2
- b. n
- c. n-1 (selected)
- d. 1

Feedback

Răspunsul corect este: n

10. Dandu-se un graf complet ponderat cu nodurile {v_1, v_2 ... v_n} astfel incat muchia (v_i, v_j) sa aiba costul 2 * |i-j|, calculati costul AMA al grafului.

Alegeți o opțiune:

a. n - 1

b. 2 * n - 2

c. n / 2

d. n ^ 2

Feedback

Răspunsul corect este: 2 * n - 2

11. În ce situații este preferabil să folosiți algoritmul Mergesort față de Quicksort care alege întotdeauna primul element ca pivot?

Alegeți o opțiune:

a. Când vectorul este (aproape) sortat crescător sau descrescător

b. Întotdeauna deoarece Mergesort este mai rapid ca Quicksort

c. Când vectorul de sortat conține mai multe duplicate

d. Niciodată, deoarece Quicksort este întotdeauna mai rapid ca Mergesort

Feedback

Răspunsul corect este: Când vectorul este (aproape) sortat crescător sau descrescător

12. Care e avantajul principal al tăieturilor alfa-beta față de Minimax clasic?

Alegeți o opțiune:

a. Este mai ușor de implementat

b. În general, returnează rezultate mai rapid

c. Returnează rezultate mai bune

d. Nu oferă niciun avantaj

Feedback

Răspunsul corect este: În general, returnează rezultate mai rapid

13. Dacă se folosește MergeSort pentru a sorta crescător vectorul [18, 52, 14, 8, 9, 2, 40, 25, 12, 17], atunci înaintea ultimei interclasări efectuate de către algoritm pentru obținerea vectorului sortat, ordinea elementelor este:

Alegeți o opțiune:

a. 2, 8, 9, 14, 18, 52, 12, 17, 25, 40

b. 14, 18, 52, 2, 8, 9, 12, 25, 40, 17

c. 8, 9, 14, 18, 52, 2, 12, 17, 25, 40

d. 8, 14, 18, 52, 2, 9, 25, 40, 12, 17

Feedback

Răspunsul corect este: 8, 9, 14, 18, 52, 2, 12, 17, 25, 40

14. Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care sunt domeniile variabilelor după aplicarea algoritmului AC1 ?

Alegeți o opțiune:

a. A = {1, 2}, B = {5, 6}, C = {5, 6}

b. A = {1, 2, 3, 4, 5}, B = {5, 6, 7}, C = {5, 6, 7}

c. A = {1, 2}, B = {5, 6, 7}, C = {5, 6, 7}

d. A = {2}, B = {5}, C = {5}

Feedback

Răspunsul corect este: A = {1, 2}, B = {5, 6}, C = {5, 6}

Subiectul 3

Un fotograf profesionist are *n* fotografii din *x* orașe. El dorește să genereze toate modurile posibile de combinații poze-orașe. Descrieți un algoritm pentru această problemă.

Rez mea: O prima solutie ce imi vine in minte este genererarea cu ajutorul unui algoritm backtracking a tuturor combinatiilor poze-orase. Mai exact problema este similara cu generarea tuturor combinarilor.

Rez lor:

Se foloseste backtracking pentru a genera toate perechile flori-orase care respecta restrictiile din enunt.

Cum se poate implementa eficient un algoritm de programare dinamică având recurența:

$$dp[n] = \begin{cases} \text{dp[n-2]} - \text{dp[n-4]}, & n \geq 4 \\ 2 * n, & n < 4 \end{cases}$$

Rez mea:

Prima idee de optimizare care imi vine in minte este exponentierea rapida pe matrici. Avem cazurile de baza:
dp[0] =0
dp[1] = 2
dp[2] = 4
dp[3] = 6
Iar apoi avem recurenta liniara dp[n] = dp[n-2] - dp[n-4]

Se construieste matricea C cu Ck-2 = 2 si Ck-4 = 4
Se foloseste ridicarea la putere in timp logaritmic si se obtine complexitate Theta(KMAX ^3 * logn), dar KMAX este 2 in cazul nostru deci Theta(logn).

Rez lor:

Exponențiere logaritmică pe matrice

Se dă un vector *v* cu *n* intervale de forma $[a_i, b_i]$. Se cere numărul maxim de intervale care se intersectează. Descrieți un algoritm eficient.

Rez mea:

Prima idee ar fi sa folsesc un algoritm de tip greedy. Pastrez intr-un vector in care tin perechie mapate, a-urile marcate cu 0, b-urile marcate cu 1.

Sortez vectorul crescator, parcurg vectorul, daca intalnesc un 0, incrementez un contor, iar daca intelesc un 1 il decrementez pentru ca am gasit un inveral ce se suprapune. Nu imi ramane decat in fiecare iteratie sa calculez un maxim dintre contor si result, unde result e initializat cu 0.

Rez lor:

Greedy: se sortează, apoi se parcurg intervalele de la stânga la dreapta. Se pornește cu un interval și cât timp se intersectează cu următorul, se aplică operația de intersecție. O(n log n).

În orașul X, rețeaua de internet este răspândita în toate cartiere, în fiecare cartier fiind amplasate mai multe routere interconectate între ele. Unele routere cedează, iar internetul pică în unul sau mai multe cartiere. Ce algoritm ați folosi pentru identificarea routerelor care duc la astfel de căderi și ce soluție sugerați pentru scăderea numărului de incidente?

Rez mea(Asta a fost de juma' de punctaj):

Ma gandesc sa folosesc Algoritmul lui Tarjan pentru a determina punctele de articulatie, iar solutia pentru scaderea numarului incidente ar fi sa elimin punctele critice.

Rez lor:

Găsim toate punctele de articulație. O soluție ar fi adăugarea de muchii redundante intre cartiere.

Se dă un graf orientat cu *m* noduri, având capacitați numere pare pe arce. Se dau o sursă *S* și *p* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la sursă la toate destinațiile.

Rez mea:

Adaug 2 noduri S'(o sursa unica) si P'(destinatie unica) cu capacitatii infinite iar apoi folosesc algoritmul Edmons-Karp pentru a determina fluxul maxim.

Rez lor:

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mari numere. Secvența este prea mare pentru a fi stocată integral in memorie.

Rez mea:

Avand in vedere ca K << N voi folosi o coada de prioritati, un minheap mai exact. Introduc in coada de prioritati primele K elemente din secventa.

Imi raman N - K elemente. Parcurg aceste elemente si daca vreunul este mai mare decat varful cozii, atunci elimin varful si introduc acest nod.

O alternativa ar fi fost bucket sort, doar ca problema nu ne permite stocarea secventei in memoriei

Rez lor:

n log k- folosim un minheap

n * k - 0.5 din punctaj